

# What Semantics Survive the Connector?

## Diagnosing VLM-to-DiT Alignment in Video Editing

Hangyu Lin\*    Chao Wen\*    Chengming Xu<sup>✉</sup>    JianXiong Gao  
 Jiangning Zhang    Xiaobin Hu    Yanwei Fu  
 HKUST    FDU    ZJU    NUS

### Abstract

Flow matching based video generative models have been increasingly relying on prepended Vision-Language Models (VLMs) to handle complex, instruction-based video editing. The prevailing assumption underlying this paradigm is that a connector module can seamlessly align the VLM’s rich multi-modal reasoning with the original text embedding space of DiTs. However, we hypothesize that this alignment acts as a severe semantic bottleneck, degrading fine-grained structural variables. Verifying this is challenging, as end-to-end evaluations conflate alignment failures with generation errors, and natural datasets lack disentangled annotations. To rigorously investigate this, we propose a controlled data processing pipeline based on video composition that results in TRACE-Edit, a diagnostic dataset focusing on relation-based editing. Leveraging this dataset, we propose a comprehensive diagnostic protocol to analyze two important designs of meta-query and connector in the existing video editing models. Systematic evaluation of four representative model cases reveals that fine-grained structural semantics can be severely degraded during alignment. Our findings overturn the assumption of lossless semantic transfer, identifying the VLM-to-DiT alignment as a major bottleneck and providing a new diagnostic foundation for future multi-modal alignment architectures.

## 1 Introduction

Modern video generative models, such as Wan [17], HunyuanVideo [10], and LTX [6], have demonstrated remarkable capabilities in synthesizing high-quality motion and appearance. However, these models cannot natively be applied to instruction-based video editing, demanding complex multi-modal reasoning, such as identifying *where* to edit, *what* attribute to apply, and *which* object must serve as a reference. Besides the limitation of pretraining tasks, the text encoders of these models often struggle with such fine-grained multi-modal binding. To this end, recent models such as UniVideo [18] and Kiwi-Edit [11] have adopted a seemingly straightforward paradigm to replace the original text encoder with a more powerful multi-modal one, such as Qwen2.5VL [1], by leveraging an MLP connector to bridge the latent spaces of these two encoders. Furthermore, these models adopt extra query tokens to enhance the condition from multi-modal instruction. The prevailing assumption underlying such a design is that the connector module can align the VLM’s rich semantic understanding with the DiT’s original condition space, based on which the query tokens provide further refinement, thereby transferring superior instruction-following capabilities.

Unfortunately, as illustrated in Fig. 1, while VLMs can successfully understand the edit instruction, the video editing models with the same VLMs fail in different ways such as routing edits to the background, dropping the edit signal entirely, or binding attributes to the wrong identity. It seems aligning the latent space of the strongly performing VLMs to their weaker counterparts may make the post-connector representations fail to preserve critical edit-related semantics and deliver compromised conditions to the DiT. Therefore it leads to an important question: *Will current design result in*

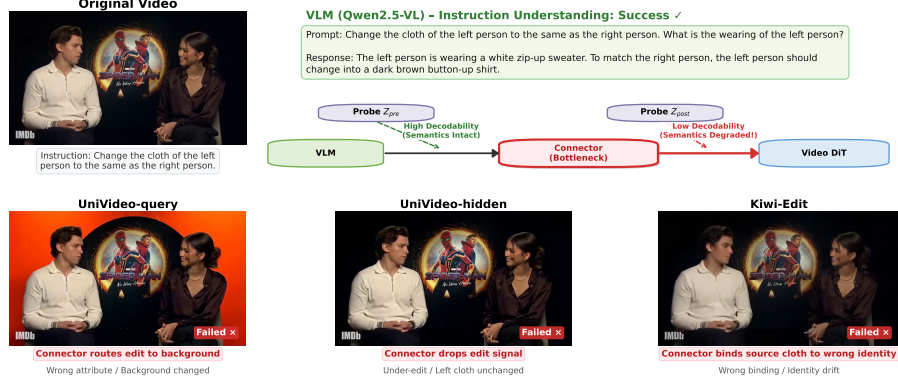


Figure 1: **The Connector Bottleneck in VLM-driven Video Editing.** Although the VLM accurately comprehends instructions (top), the alignment connector degrades rich structural semantics. This compromised conditioning causes downstream Video DiT failures, including misrouted attributes, dropped signals, and identity drift (bottom).

*semantic bottleneck?* Verifying and isolating such bottleneck is challenging, as existing end-to-end video editing benchmarks often conflate these alignment failures with the DiT’s own denoising or attention errors, while overlooking the need for disentangled probing.

To rigorously investigate this, we introduce TRACE-Edit, a controlled diagnostic benchmark and evaluation protocol specifically focusing on analyzing the VLM-to-DiT alignment. Rather than relying on unstructured natural videos, TRACE-Edit employs a factorized relation-editing protocol. We first synthesize single-object atomic videos with specific attributes, which are verified and filtered with VLMs, then randomly compose them into video grids. By pairing these composed videos with instructions that specify edited and reference slots, a large-scale synthetic video editing benchmark with structural instructions can be easily built at low costs. Crucially, every example is equipped with exact ground-truth labels for slots, attributes, and object roles. TRACE-Edit is not meant to replace natural video editing benchmarks, but to isolate hidden failure modes that natural benchmarks conflate. Leveraging this benchmark, we propose a diagnostic framework to probe the internal properties of VLM-to-DiT alignment, specifically condition space geometry, semantic accessibility, token-level evidence routing, and attention allocation. Our findings overturn the prevailing assumption that connectors are semantic-preserving adapters. Although coarse intent survives, fine-grained structural semantics severely degrade during alignment (e.g., target-slot decodability drops by 18.4% in U-Query). Crucially, token-route analysis reveals a severe misalignment regarding query tokens: while text tokens retain strong semantics, alignment drastically shifts actual semantic evidence away from query tokens, yet the DiT still allocates dominant attention to them. This disconnect between information routing and attention allocation destroys slot–value–role bindings. Consequently, edit result judgment performed by VLMs confirm that structural misassignments (accounting for up to 61.9% of errors), rather than conservative under-editing, constitute the primary failure mode, exposing the connector as a major semantic bottleneck. In summary, our main contributions are:

1. We introduce TRACE-Edit, a controlled relation-editing benchmark that isolates slot, value, and object-role variables via synthetic video composition, providing precise labels for diagnosing structural semantics.
2. We propose a comprehensive diagnostic protocol utilizing specifically for the VLM-to-DiT alignment.
3. We present a rigorous empirical study demonstrating that current VLM-to-DiT designs suffer from semantic degradation, thus identifying the connector as a primary bottleneck in video editing models.

## 2 Related Work

**VLM-related Video Diffusion Models** A growing family of unified multi-modal generation systems couples an autoregressive VLM/MLLM with a diffusion or DiT decoder. MetaQueries introduce learnable query tokens as the bridge from MLLM latents to a diffusion generator [15]. Query-Kontext uses multi-modal “kontext” tokens to bridge a VLM and diffusion model for image generation

and editing [16]. BLIP3-o and ILLUME+ explore hybrid multi-modal understanding-generation architectures with diffusion components [3, 7], while OmniGen shows that unified image generation can be driven by instruction-like multi-modal prompts [19]. Recent video-oriented systems such as UniVideo, Kiwi-Edit, OmniWeaving, InstructX, and VINO extend the trend to video generation and editing by combining VLM-side reasoning with DiT-side synthesis [4, 11, 13, 14, 18].

**Video Editing Benchmarks** Recent benchmarks have significantly advanced the evaluation of text-guided generation and editing. Works such as EditVal [2], VBench [8], and EvalCrafter [12] provide comprehensive evaluation suites for image editing and video generative models. Building on this, newer datasets like EditVerseBench [9] and IVEBench [5] have further modernized the landscape by introducing complex instruction-guided taxonomies and MLLM-based metrics for video editing. These benchmarks rely on unstructured, in-the-wild videos that lack disentangled annotations for spatial slots and relational bindings. In contrast, TRACE-Edit is explicitly designed as a diagnostic benchmark specifically for probing intermediate features and pinpointing the exact semantic bottlenecks in VLM-to-DiT alignment pathways, a capability absent in prior benchmarks.

### 3 The TRACE-Edit Diagnostic Benchmark

#### 3.1 Task Formulation and Variables

TRACE-Edit is specifically designed to make structural editing semantics observable and quantifiable. A single example in our benchmark consists of a short composite video  $x$ , a natural language instruction  $I$ , and a comprehensive label set:

$$y = (y_{\text{attr}}, y_{\text{slot}}, y_{\text{tgt}}, y_{\text{editobj}}, y_{\text{refobj}})$$

To break this down:

- $y_{\text{attr}}$ : The attribute type being edited (color, material, or action).
- $y_{\text{slot}}$ : The spatial slot of the object to be edited.
- $y_{\text{tgt}}$ : The target value inherited from the reference object.
- $y_{\text{editobj}}/y_{\text{refobj}}$ : The names of the edited and reference objects, respectively.

**Relational Labels and Grid Layout.** We adopt a four-slot grid layout (top-left, top-right, bottom-left, bottom-right). This deliberately raises the entropy of the “where” decision from a simple binary (left/right) to a four-way choice, while keeping the visual structure clean enough for controlled probing. As illustrated in Fig. 2, a typical relation instruction asks the editor to modify one cell based on another. Given the instruction “Change the material of the object in the top-left position to match the object in the bottom-right position,” the benchmark extracts the structured relational labels  $y$ . Specifically, it identifies the attribute to be modified ( $y_{\text{attr}} = \text{material}$ ), the spatial location of the target video ( $y_{\text{slot}} = \text{top-left}$ ), and the target attribute value derived from the reference video ( $y_{\text{tgt}} = \text{plastic}$ ). Furthermore, it grounds the object categories for both slots, identifying the edited object as  $y_{\text{editobj}} = \text{vase}$  and the reference object as  $y_{\text{refobj}} = \text{cup}$ .

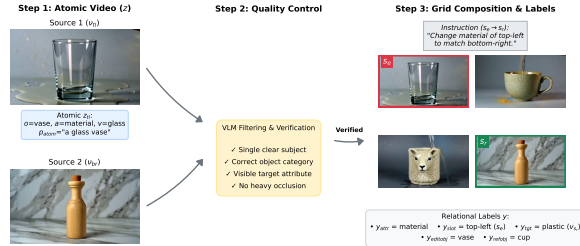


Figure 2: Pipeline for constructing TRACE-Edit.

**Target Formulation.** Formally, let  $S = \{\text{tl}, \text{tr}, \text{bl}, \text{br}\}$  be the slot set. A composite scene contains cell metadata  $G = \{c_s = (o_s, a, v_s, \nu_s) : s \in S\}$ , where  $o_s$  is the object,  $a$  is the shared attribute type,  $v_s$  is the slot-specific value, and  $\nu_s$  is the underlying atomic video. When we sample an edited slot  $s_e$  and a distinct reference slot  $s_r$ , the target label deterministically becomes  $y_{\text{tgt}} = v_{s_r}$  and  $y_{\text{slot}} = s_e$ . This strict formulation allows us to evaluate whether a model preserves the exact semantic variables needed for correct editing, rather than merely generating a visually plausible video.

### 3.2 Benchmark Construction Pipeline

To ensure high-quality compositions and precise control, TRACE-Edit is built from the ground up through a rigorous pipeline, starting from single-object videos and culminating in complex relational grids.

**Atomic Video Generation.** We generate 81-frame clips at  $480 \times 832$  resolution for each atomic description using the Wan2.2-A14B text-to-video model [17]. An atomic instance is defined as:

$$z = (u, o, a, v, p_{\text{atom}}, \nu) \quad (1)$$

where  $u$  is a unique identifier,  $o$  is drawn from a 15-object pool,  $a \in \{\text{color, material, action}\}$ ,  $v$  is the attribute value,  $p_{\text{atom}}$  is the generation prompt, and  $\nu$  is the generated video. Our current instantiation features 9 colors, 10 materials, and 9 actions. To facilitate fine-grained generation, the system parameterizes the source videos into these atomic variables  $z$ . Returning to the example in Fig. 2, for the top-left source video, the benchmark identifies the base object  $o = \text{vase}$  and its original material  $v = \text{glass}$ , constructing the atomic prompt  $p_{\text{atom}} = \text{“a glass vase”}$ . Similarly, for the bottom-right reference video, it extracts  $o = \text{cup}$  and  $v = \text{plastic}$ , yielding the prompt  $p_{\text{atom}} = \text{“a plastic cup”}$ . These atomic prompts are generated via compositional natural-language templates (e.g., adjectives for color/material, motion phrases for actions) wrapped in strict constraints to ensure a single clear subject and stable camera motion.

**Quality Control via VLM Filtering.** Crucially, before any composition occurs, we apply a strict Vision-Language Model (VLM) verification stage. The VLM acts as an automated quality inspector, answering a structured questionnaire to verify: (1) the presence of a single central subject, (2) the correct object category  $o$ , and (3) the visibility of the attribute value  $v$ . Only videos that pass all checks are admitted into the composition pool. This atomic-level filtering is vital: it guarantees that any failure in the final relational editing task is due to the model’s inability to understand relational binding, rather than a hallucination or rendering failure in the underlying source videos.

**Grid Composition and Relational Direction.** Verified atomic videos are grouped by attribute type. For a single composite scene, we select one attribute type and draw four distinct values, randomly assigning four corresponding atomic videos to the ordered grid slots  $S$ . The videos are spatially resized into tiles, pasted into their respective slots, and temporally synchronized. A key design feature of TRACE-Edit is its sensitivity to **relational direction**. From a single composite scene, we sample an edited slot  $s_e$  and a reference slot  $s_r$ , and generate two directed instructions: one forward ( $s_e \rightarrow s_r$ ) and one inverted ( $s_r \rightarrow s_e$ ). Both instructions share the exact same visual input but require completely different target values and object roles. A model that merely recognizes the scene without understanding directional instructions will fail this test. Moreover, our benchmark can be easily extended to more complicated version by compositing more video grids, which can be useful for more powerful video editing models in the future.

**Instruction Generation and Dataset Scale.** Instructions are instantiated using attribute-specific templates (e.g., matching colors/materials, or following motion trajectories) and include an evaluation query designed to verify the edit, whose ground-truth answer is  $v_{s_r}$ . As summarized in Tab. 1, TRACE-Edit operates at a significant scale, containing 5,524 unique composite videos yielding 11,048 relation-editing examples. Because each composite video provides both a forward and an inverted instruction, the dataset is perfectly balanced to diagnose true relational comprehension across color (6,490 examples), material (3,096), and action (1,462) domains. Detailed object/value pools, atomic prompt templates, VLM verifier prompts, and relation-row metadata are provided in Append. C.

| Quantity            | Value             |
|---------------------|-------------------|
| Composite videos    | 5,524             |
| Relation examples   | 11,048            |
| Relation directions | 5,524/5,524       |
| Slot labels         | 4 classes         |
| Attribute examples  | 6,490/3,096/1,462 |
| Semantic labels     | 5 groups          |

Table 1: Summary of TRACE-Edit.

## 4 Experimental Protocol

Our experiments are organized as connector diagnostics rather than a video-editing leaderboard. For each evaluated model, we compare the VLM representation before and after connector alignment, and ask whether the task-critical information defined by TRACE-Edit remains accessible. This framing isolates the connector as a semantic transfer module: the goal is not to rank editors by final

video quality, but to test whether the representation consumed by the video DiT still exposes enough condition-related information needed for the edit.

Tab. 2 summarizes the four models utilized in our experiments. U-Hidden and U-Query represent UniVideo based on HunyuanVideo and transfer editing condition through vision-and-text and text-and-query routes, respectively. Kiwi-Edit represents a query-based video editor built on Wan2.2 5B. Wan-Query is a model trained by us, which shares the same condition embedding mechanism as U-Query to embed the editing condition, but instead adopts Wan2.1 14B as DiT backbone and uses the condition embedding through cross-attention. We follow UniVideo and Kiwi-Edit to perform a multi-stage training strategy, i.e. (1) train the connector with T2V and I2V data, (2) train connector and DiT parameters with T2V and I2V data, and (3) train connector and DiT parameters with video editing data. Although differences in training data preclude a fair comparison of final editing quality, including Wan-Query allows us to isolate the impact of connector and query designs, ruling out the confounding factor of a weak DiT backbone.

For each relation-editing example, we extract a pre-connector feature  $h_{\text{pre}}^{(i)}$  and the corresponding post-connector DiT-facing feature  $h_{\text{post}}^{(i)} = \mathcal{C}(h_{\text{pre}}^{(i)})$ . Depending on the model, these features are pooled over hidden states, text-plus-query tokens, or learnable query tokens. All probing experiments use composition-level train/validation/test splits, so that no four-cell source scene appears in both training and testing. Results are averaged over 3 seeds.

| Model     | DiT backbone | Condition route |
|-----------|--------------|-----------------|
| U-Hidden  | HunyuanVideo | text+vision     |
| U-Query   |              | text+query      |
| Kiwi-Edit | Wan2.2 5B    | query           |
| Wan-Query | Wan2.1 14B   | text+query      |

Table 2: Model used for diagnosis.

The diagnostics below answer five questions: (1) which variables survive connector alignment, (2) whether alignment is geometrically non-trivial, (3) how query evidence is redistributed, (4) how the DiT consumes condition tokens, and (5) whether generated-video failures concentrate on the same structural axes.

## 5 Empirical Analysis of VLM-to-DiT Semantic Preservation

### 5.1 Geometric Reconfiguration of Post-connector Representations

**Experimental setup and metrics.** This diagnostic tests whether connector alignment merely projects VLM features into the original DiT text embedding space or substantially reconfigures their global geometry. For each model, we extract the pre-connector embeddings and their corresponding post-connector DiT embeddings from all samples to form the feature matrices  $X, Y \in \mathbb{R}^{n \times d}$ . For a centered feature matrix  $X$  with singular values  $\{\sigma_k\}$ , we define the normalized singular mass  $p_k = \sigma_k / \sum_j \sigma_j$  and the effective rank as  $r_{\text{eff}}(X) = \exp(-\sum_k p_k \log p_k)$ . We also measure feature variance and linear CKA (Centered Kernel Alignment) between the centered matrices  $X$  and  $Y$  as follows:

$$\text{Var}(X) = \frac{1}{nd} \sum_{i=1}^n \sum_{j=1}^d (X_{ij} - \bar{X}_j)^2, \quad \text{CKA}(X, Y) = \frac{\|X^\top Y\|_F^2}{\|X^\top X\|_F \|Y^\top Y\|_F}, \quad (2)$$

where the effective rank measures the spread of singular mass across feature dimensions, feature variance measures representation scale, and CKA measures global similarity between the pre- and post-connector condition spaces.

**Results and analysis.** Tab. 3 shows that connector alignment substantially reconfigures the condition space. U-Hidden and U-Query undergo large effective-rank reductions. In contrast, Kiwi-Edit and Wan-Query show an increase in effective rank after alignment, yet both exhibit a severe collapse in feature variance. These findings establish that connector alignment is a profound geometric transformation rather than a mere passive format conversion. However, global geometry alone cannot determine which semantic variables survive: models with similar CKA or rank trends can differ sharply in their preservation of slot, value, and role information. This discrepancy motivates the variable-level semantic decodability in Sec. 5.2 and the subsequent token-route analyses.



| Model     | Effective rank |        |          | Feature variance      |                       |          | CKA  |
|-----------|----------------|--------|----------|-----------------------|-----------------------|----------|------|
|           | Pre            | Post   | $\Delta$ | Pre                   | Post                  | $\Delta$ |      |
| U-Hidden  | 1096.1         | 458.2  | -58.2%   | $3.25 \times 10^{-1}$ | $6.17 \times 10^{-2}$ | -81.0%   | .840 |
| U-Query   | 871.3          | 350.5  | -59.8%   | $4.27 \times 10^{-2}$ | $4.20 \times 10^{-3}$ | -90.2%   | .896 |
| Kiwi-Edit | 748.9          | 824.8  | +10.1%   | $5.26 \times 10^{-4}$ | $1.68 \times 10^{-5}$ | -96.8%   | .966 |
| Wan-Query | 892.0          | 1041.2 | +16.7%   | $3.77 \times 10^{-1}$ | $1.00 \times 10^{-3}$ | -99.7%   | .871 |

Table 3: Representation geometry before and after connector alignment. Connector alignment substantially changes scale and rank structure for every model, showing that DiT-facing condition representations are not passive projections of VLM-side features.

| Variable          | Chance | U-Hidden    | U-Query     | Kiwi-Edit   | Wan-Query   |
|-------------------|--------|-------------|-------------|-------------|-------------|
| Attribute type    | 1/3    | .988 → .977 | .998 → .998 | .999 → .997 | .999 → .999 |
| Edited slot       | 1/4    | .260 → .248 | .923 → .739 | .992 → .987 | .897 → .863 |
| Target value avg. | ≈ .107 | .220 → .214 | .517 → .491 | .502 → .485 | .345 → .306 |
| Edited object     | 1/15   | .340 → .341 | .375 → .426 | .470 → .420 | .462 → .471 |
| Reference object  | 1/15   | .343 → .342 | .415 → .444 | .586 → .560 | .355 → .345 |

Table 4: Pre-connector → post-connector linear-probe accuracy on TRACE-Edit. Target value is the unweighted average over action, material, and color target probes; its chance level is the corresponding average over the attribute-specific label spaces. Full per-attribute target results are in Appendix F.

## 5.2 Selective Preservation of Relational Variables under Connector Alignment

**Experimental setup and metrics.** This diagnostic measures whether each task-critical relational information remains linearly accessible before and after connector alignment. We cover five information types including the type of edit attribute, edited slot, target value, edited object and reference object in this experiment. For each label family  $y$  and representation stage  $r \in \{\text{pre}, \text{post}\}$ , we train a linear classifier

$$f_y^r(h) = \text{softmax}(W_y^r \phi_r(h) + b_y^r), \quad (3)$$

where  $\phi_r(\cdot)$  denotes standardized PCA features with dimension 128 fit on the training set. The probes are optimized by SGD with learning rate 0.1, weight decay  $10^{-4}$ , momentum 0.9. We adopt accuracy as the main metric. Since target-value label spaces differ by attribute type, the probes for target attribute value for three attribute types are trained respectively, based on which the average metric is reported.

**Results and analysis.** Tab. 4 demonstrates a selective preservation pattern. (1) Performance on the attribute-type task remains near-perfect after alignment across all models. Thus, connectors reliably preserve the coarse edit category, e.g., whether an instruction concerns color, material, or action. (2) Results from the edited-slot task separate the models into distinct regimes. While all four models exhibit decreased accuracy with post-connector embeddings, U-Hidden fails entirely on this task, yielding near-random performance with both pre- and post-connector features. U-Query exposes the edited slot before alignment, but the post-connector embedding loses a large fraction of that accessibility, dropping from .923 to .739. Other two models shows a smaller but consistent decline. (3) Target-value semantics are more fragile. Even models with strong slot prediction ability lose target-value accessibility after alignment. (4) Object-role information does not follow a uniform degradation pattern: Kiwi-Edit loses edited-object and reference-object decodability, while U-Query gains object-role accessibility after alignment. This strengthens our hypothesis that connector alignment does not simply erase all fine-grained information, but rather fractures the relational edit program by **weakening and redistributing specific semantic components**. While capable of preserving coarse edit intent, the connector fails to reliably preserve the fine-grained slot–value–role binding required by our benchmark.

## 5.3 Token-Route Decomposition of Query Semantics

**Experimental setup and metrics.** This diagnostic investigates where semantic evidence resides within models utilizing extra query tokens. Concretely, we decompose U-Query into three feature views: a **mixed** view containing embeddings of both text and query tokens, a **text-only** view, and

a **query-only** view. We then measure edited-slot and target-value probing ability from each view using the same methodology as in Sec. 5.2. To localize evidence within the mixed route, we compute token-level margin contributions for the edited-slot classifier. For a token  $t$ , correct class  $c^*$ , and strongest incorrect class  $\hat{c}$ , the approximate margin contribution is

$$m_{it} = \langle w_{c^*} - w_{\hat{c}}, \psi(h_{it}) \rangle, \quad (4)$$

where  $\psi(h_{it})$  is the token contribution after the same linear feature map. We report the total margin  $\sum_t m_{it}$ , normalized positive-contribution entropy

$$H_i^+ = -\frac{1}{\log T} \sum_{t=1}^T q_{it} \log q_{it}, \quad q_{it} = \frac{\max(m_{it}, 0)}{\sum_j \max(m_{ij}, 0)}, \quad (5)$$

top-1 positive mass  $\max_t q_{it}$ , and the fraction of positive mass assigned to query tokens.

| View       | Attr. type                | Edited slot               | Material target           | Color target              | Reference object          |
|------------|---------------------------|---------------------------|---------------------------|---------------------------|---------------------------|
| Mixed      | 0.998 $\rightarrow$ 0.998 | 0.923 $\rightarrow$ 0.739 | 0.619 $\rightarrow$ 0.593 | 0.675 $\rightarrow$ 0.633 | 0.414 $\rightarrow$ 0.444 |
| Text only  | 1.000 $\rightarrow$ 0.999 | 0.999 $\rightarrow$ 0.988 | 0.897 $\rightarrow$ 0.829 | 0.924 $\rightarrow$ 0.868 | 0.813 $\rightarrow$ 0.595 |
| Query only | 0.999 $\rightarrow$ 0.994 | 0.781 $\rightarrow$ 0.556 | 0.496 $\rightarrow$ 0.528 | 0.551 $\rightarrow$ 0.512 | 0.385 $\rightarrow$ 0.389 |

Table 5: U-Query feature-view ablation.

**Results and analysis.** The ablation results (Table 5) demonstrate that a query-based model contains multiple semantic routes with varying sensitivities to connector alignment. In U-Query, text tokens retain strong instruction semantics: the edited-slot accuracy remains high (0.999  $\rightarrow$  0.988), alongside material (0.897  $\rightarrow$  0.829) and color (0.924  $\rightarrow$  0.868) targets. Conversely, query-only features are weaker and more fragile to connector alignment, with edited-slot accuracy dropping sharply (0.781  $\rightarrow$  0.556). The mixed route lies between these extremes, dropping from 0.923 to 0.739 on the edited-slot task. This indicates that the connector does not universally erase slot semantics from every subview; rather, it alters how the mixed route exposes this information.

Furthermore, token-level margins reveal this routing shift at a finer scale. In U-Query, the mean edited-slot margin drops from 18.08 to 11.13 after alignment. The positive evidence mass contributed by query tokens decreases from 0.874 to 0.623, while the top-1 positive mass spikes from 0.016 to 0.291. This indicates a shift toward a highly concentrated, less query-dominated evidence pattern after alignment. Cross-model comparisons reinforce this interpretation: Kiwi-Edit maintains a stable edited-slot margin (24.82  $\rightarrow$  27.70), consistent with its strong slot preservation, whereas Wan-Query shows only a mild edited-slot margin drop (16.57  $\rightarrow$  15.99) but still loses target-value decodability. Therefore, while query-token conditioning provides a pathway for relational evidence, its preservation heavily depends on whether the connector alignment maintains the accessibility of slot–value–role information along that specific route.

#### 5.4 Attention Routing and Semantic Consumption by the DiT

**Experimental setup and metrics.** This diagnostic tests how the DiT consumes the post-connector embeddings. For each model, we focus on the attention from generated video tokens to condition tokens. Formally, for attention weight  $\alpha_{\ell,t,h,q,k}$  from generated token  $q$  to condition token  $k$  at layer  $\ell$ , denoising step  $t$ , and head  $h$ , the group attention share for condition-token group  $g$  is

$$A_g = \frac{\sum_{\ell,t,h,q} \sum_{k \in g} \alpha_{\ell,t,h,q,k}}{\sum_{\ell,t,h,q} \sum_{k \in \mathcal{K}} \alpha_{\ell,t,h,q,k}}. \quad (6)$$

We also normalize the aggregated condition-token distribution  $\bar{\alpha}_k$ , and report entropy  $H_A = -\sum_k \bar{\alpha}_k \log \bar{\alpha}_k$ , top- $K$  mass  $M_K = \sum_{k \in \text{Top}K(\bar{\alpha})} \bar{\alpha}_k$ , and head agreement, measured as the average Jaccard similarity between the top-16 condition-token sets selected by different heads. Statistics are computed on 20 examples per model. U-Hidden is not included in this experiment due to its lack of query tokens.

**Results and analysis.** Fig. 3 reveals three ways in which the DiT consumes post-connector embeddings. First, U-Query exhibits a mixed handoff regime: overall condition attention remains text-dominant (text .573, query .427), while query share increases from .384 early in denoising to

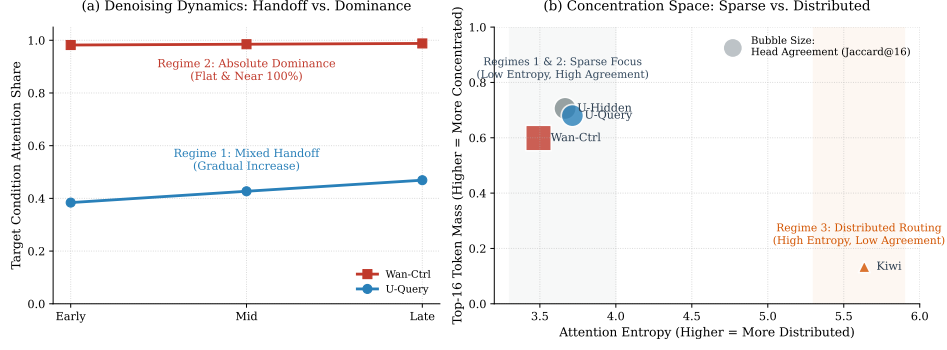


Figure 3: Condition-attention routing regimes. U-Query gradually increases query attention while retaining text anchors; Wan-Query is query-dominated from the beginning; Kiwi-Edit distributes attention over many learnable queries with low head agreement.

| Model     | Evaluated<br>$n_{eval}$ | PassRate<br>Pass | StructErr<br>Wrong slot + binding | UnderEdit<br>Under-edit |
|-----------|-------------------------|------------------|-----------------------------------|-------------------------|
| U-Hidden  | 119                     | 25.2%            | 58.0%                             | 12.6%                   |
| U-Query   | 139                     | 22.3%            | 61.9%                             | 12.2%                   |
| Kiwi-Edit | 137                     | 38.0%            | 40.9%                             | 14.6%                   |
| Wan-Query | 135                     | 25.2%            | 57.0%                             | 11.1%                   |

Table 6: Output failure evaluation on 144 examples per model. Rates are computed among successfully evaluated outputs. Structural errors dominate under-editing across all models.

.469 late in denoising. The DiT therefore allocates increasing attention to query tokens while retaining text as a structural anchor. Second, Wan-Query exhibits absolute query dominance. Although it contains both text and query condition groups, almost all condition attention is assigned to query tokens: query share is .985 overall and changes only slightly from early to late denoising. Its attention is also relatively concentrated, with top-32 condition tokens absorbing .977 of the condition attention and a higher head top-16 Jaccard score than U-Query. Third, Kiwi-Edit uses distributed query routing: its condition group consists of learnable queries, but attention is spread across them, resulting in high entropy (5.635), low top-16 mass (.134), and low head agreement (.116). These regimes show that attention allocation and semantic preservation are not equivalent. Wan-Query assigns almost all condition attention to query tokens, yet still loses target-value accessibility and exhibits structural output failures. The relevant question is therefore not only how much attention the DiT assigns to query tokens, but whether that route preserves and exposes the correct slot–value–role information.

## 5.5 Output-Level Structural Failure Analysis

**Experimental setup and metrics.** This diagnostic connects connector-related measurements to the generated videos. Because full video inference is expensive, we evaluate a balanced subset of 144 examples for each model. Each generated output is assigned to one of six categories: pass (i.e. the edit is success), under-edit (i.e. the source video is not changed), wrong slot, wrong target value, wrong object/binding, or undetermined. Let  $n_{eval}$  be the number of evaluated outputs,  $n_{pass}$  the number of correct outputs,  $n_{slot}$  the number of wrong-slot outputs,  $n_{bind}$  the number of wrong-object/binding outputs, and  $n_{under}$  the number of under-edited outputs. We report

$$\text{PassRate} = \frac{n_{pass}}{n_{eval}}, \quad \text{StructErr} = \frac{n_{slot} + n_{bind}}{n_{eval}}, \quad \text{UnderEdit} = \frac{n_{under}}{n_{eval}}. \quad (7)$$

Structural error is defined as wrong-slot plus wrong-object/binding failures. Such an experiment serves as a consistency check between representation diagnostics and output behavior.

**Results and analysis.** Tab. 6 shows that under-editing is not the dominant failure mode. Across models, under-editing accounts for only 11.1%–14.6% of evaluated outputs, whereas structural errors account for 40.9%–61.9%. The generated failures are therefore primarily structural rather than purely conservative: the systems often perform an edit, but fail to preserve the assignment specified by the instruction, i.e. which slot should change, which object should be edited, and which reference object or target value should govern the edit. The cross-model pattern aligns with the connector-side



diagnostics without requiring a per-sample causal claim. Specifically, U-Query exhibits the highest structural-error rate, consistent with its severe post-connector slot degradation. Meanwhile, Kiwi-Edit achieves the highest pass rate and the lowest structural-error rate, consistent with stronger slot preservation; however, it still produces numerous structural failures because target-value and role variables remain fragile. Furthermore, Wan-Query performs similarly to the two UniVideo variants despite using a Wan backbone rather than HunyuanVideo, supporting the conclusion that backbone replacement alone does not remove the VLM-to-DiT alignment bottleneck. These output-level results connect the representation diagnostics to end-task behavior: the principal failure mode is structural misassignment rather than a generic lack of editing strength.

## 6 Discussion

**Connectors are critical semantic transferers.** The central lesson is that VLM-to-DiT alignment should not be evaluated only by end-to-end video quality. A connector may preserve coarse editing categories while failing to expose the variables that make an edit correct. One likely reason is that current training aligns VLM features to a pre-existing DiT text-conditioning space, which was not originally optimized to encode disentangled relational edit programs such as edited slots, target values, and object roles. Another possible contributor is supervision mismatch: generic T2V, I2V, and editing data may support coarse instruction following, but may not sufficiently constrain fine-grained slot-value-role preservation. In TRACE-Edit, attribute type is almost perfectly preserved, but edited slot, target value, and object-role variables behave differently across models. This is why output metrics alone are insufficient: two systems can both understand that an instruction asks for a material edit while differing sharply in whether they edit the correct cell, transfer the correct value, or bind the correct reference object.

**Query token conditioning routes help, but introduce a routing problem.** Query tokens are not a semantic solution by themselves. They provide a route through which text-side, visual, and relational evidence can reach the DiT, but the connector must still preserve and expose the right variables along that route. U-Query demonstrates the failure case: slot semantics are available before alignment but weakened in the DiT-facing mixed view. Kiwi-Edit demonstrates the partial success case: slot semantics can be preserved, yet target-value and role-binding semantics can still degrade. The design problem is therefore not merely whether to add queries, but how to make the query route preserve the whole slot-value-role information.

**Backbone replacement is not enough.** Since Wan-Query retains a UniVideo-style VLM utilization but uses a Wan DiT backbone apart from HunyuanVideo, its persistent target-value degradation shows that a stronger DiT cannot recover structural variables that the model fails to expose. This does not mean the backbone is irrelevant, but rather means that backbone quality and the connector’s ability of semantic preservation are distinct requirements for a qualified video editing model.

**What future video editors should report.** VLM-conditioned video editors that adopt the connector alignment strategy should report connector related diagnostics for task-critical variables, not only output-level instruction-following or video quality scores. Moreover, if a model relies on query tokens, performance focusing on whether slot, value, and object-role variables remain accessible after the query connector is also necessary for comprehensive analysis of the model. These diagnostics are inexpensive compared with full video inference and can reveal structural failure modes before large-scale generation studies.

## 7 Conclusion

We introduced TRACE-Edit as a systematic diagnostic tool for VLM-to-DiT video editing models. Across various controlled systems, connectors preserve coarse attribute categories but degrade fine-grained structural semantics such as edited slots, target values, and object roles. These representation-level losses align with output-level structural failures. Future VLM-conditioned video editors should therefore treat connector design as a semantic-preserving problem and connector-side diagnostics can be taken into account as critical experiment metrics in addition to final generation quality.

## References

- [1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [2] Samyadeep Basu, Mehrdad Saberi, Shweta Bhardwaj, Atoosa Malemir Chegini, Daniela Massiceti, Maziar Sanjabi, Shell Xu Hu, and Soheil Feizi. Editval: Benchmarking diffusion based text-guided image editing methods. *arXiv preprint arXiv:2310.02426*, 2023.
- [3] Jiu hai Chen, Zhiyang Xu, Xichen Pan, Yushi Hu, Can Qin, Tom Goldstein, Lifu Huang, Tianyi Zhou, Saining Xie, Silvio Savarese, Le Xue, Caiming Xiong, and Ran Xu. Blip3-o: A family of fully open unified multimodal models—architecture, training and dataset. *arXiv preprint arXiv:2505.09568*, 2025.
- [4] Junyi Chen, Tong He, Zhoujie Fu, Pengfei Wan, Kun Gai, and Weicai Ye. VINO: A unified visual generator with interleaved omnimodal context. *arXiv preprint arXiv:2601.02358*, 2026.
- [5] Yinan Chen, Jiangning Zhang, Teng Hu, Yuxiang Zeng, Zhucun Xue, Qingdong He, Chengjie Wang, Yong Liu, Xiaobin Hu, and Shuicheng Yan. IVEBENCH: Modern benchmark suite for instruction-guided video editing assessment. *arXiv preprint arXiv:2510.11647*, 2025.
- [6] Yoav HaCohen, Benny Brazowski, Nisan Chiprut, Yaki Bitterman, Andrew Kvochko, Avishai Berkowitz, Daniel Shalem, Daphna Lifschitz, Dudu Moshe, Eitan Porat, et al. Ltx-2: Efficient joint audio-visual foundation model. *arXiv preprint arXiv:2601.03233*, 2026.
- [7] Runhui Huang, Chunwei Wang, Junwei Yang, Guansong Lu, Yunlong Yuan, Jianhua Han, Lu Hou, Wei Zhang, Lanqing Hong, Hengshuang Zhao, and Hang Xu. Illume+: Illuminating unified mllm with dual visual tokenization and diffusion refinement. *arXiv preprint arXiv:2504.01934*, 2025.
- [8] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21807–21818, 2024.
- [9] Xuan Ju, Tianyu Wang, Yuqian Zhou, He Zhang, Qing Liu, Nanxuan Zhao, Zhifei Zhang, Yijun Li, Yuanhao Cai, Shaoteng Liu, et al. Editverse: Unifying image and video editing and generation with in-context learning. *arXiv preprint arXiv:2509.20360*, 2025.
- [10] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, et al. Hunyuanvideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.
- [11] Yiqi Lin, Guoqiang Liang, Ziyun Zeng, Zechen Bai, Yanzhe Chen, and Mike Zheng Shou. Kiwi-edit: Versatile video editing via instruction and reference guidance. *arXiv preprint arXiv:2603.02175*, 2026.
- [12] Yaofang Liu, Xiaodong Cun, Xuebo Liu, Xintao Wang, Yong Zhang, Haoxin Chen, Yang Liu, Tiejiong Zeng, Raymond Chan, and Ying Shan. EvalCrafter: Benchmarking and evaluating large video generation models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 22139–22149, 2024.
- [13] Chong Mou, Qichao Sun, Yanze Wu, Pengze Zhang, Xinghui Li, Fulong Ye, Songtao Zhao, and Qian He. InstructX: Towards unified visual editing with mllm guidance. <https://arxiv.org/abs/2510.08485>, 2025.
- [14] Kaihang Pan, Qi Tian, Jianwei Zhang, Weijie Kong, Jiangfeng Xiong, Yanxin Long, Shixue Zhang, Haiyi Qiu, Tan Wang, Zheqi Lv, Yue Wu, Liefeng Bo, Siliang Tang, and Zhao Zhong. Omniweaving: Towards unified video generation with free-form composition and reasoning. <https://arxiv.org/abs/2603.24458>, 2026.
- [15] Xichen Pan, Satya Narayan Shukla, Aashu Singh, Zhuokai Zhao, Shlok Kumar Mishra, Jialiang Wang, Zhiyang Xu, Jiu hai Chen, Kunpeng Li, Felix Juefei-Xu, Ji Hou, and Saining Xie. Transfer between modalities with metaqueries. *arXiv preprint arXiv:2504.06256*, 2025.
- [16] Yuxin Song, Wenkai Dong, Shizun Wang, Qi Zhang, Song Xue, Tao Yuan, Hu Yang, Haocheng Feng, Hang Zhou, Xinyan Xiao, and Jingdong Wang. Query-kontext: An unified multimodal model for image generation and editing. *arXiv preprint arXiv:2509.26641*, 2025.
- [17] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

- [18] Cong Wei, Quande Liu, Zixuan Ye, Qiulin Wang, Xintao Wang, Pengfei Wan, Kun Gai, and Wenhui Chen. Univideo: Unified understanding, generation, and editing for videos. *arXiv preprint arXiv:2510.08377*, 2026.
- [19] Shitao Xiao, Yueze Wang, Junjie Zhou, Huaying Yuan, Xingrun Xing, Ruiran Yan, Chaofan Li, Shutong Wang, Tiejun Huang, and Zheng Liu. Omnigen: Unified image generation. *arXiv preprint arXiv:2409.11340*, 2024.

## A Limitations

Our analysis focuses on connector-based VLM-to-DiT video editing models, where a pre-trained or independently trained VLM representation is aligned with the conditioning space of a video DiT through an explicit connector. This setting covers the design pattern adopted by the evaluated systems and directly matches our goal of diagnosing whether fine-grained relational semantics survive the connector. However, our experiments do not cover architectures that are natively trained with VLM-based conditioning throughout the video generator, or systems in which multimodal reasoning and denoising are jointly optimized without an explicit post-hoc VLM-to-DiT alignment strategy, since such models are currently not accessible. Moreover, our proposed TRACE-Edit deliberately sacrifices natural-scene diversity for causal control over slot, value, and role variables. The video grid design is not intended to approximate all real editing scenarios but rather a stress test for relational binding under known ground truth.

## B Additional Benchmark and Protocol Details

**TRACE-Edit composition.** Each TRACE-Edit composite video is assembled from four verified atomic videos. A composite instance uses one attribute type and four different attribute values whenever possible. We draw an edited slot and a reference slot, store the object identity and attribute value for each cell, and generate two relation directions when applicable. This yields 5,524 composite videos and 11,048 relation-editing examples.

**Probe configuration.** Unless otherwise stated, linear-probe analysis probes use PCA dimension 128, SGD with learning rate 0.1, weight decay  $10^{-4}$ , momentum 0.9, patience 30, and train/validation/test ratio 0.7/0.15/0.15. The reported mean and standard deviation are over split seeds 0, 1, and 42.

**Computational resources.** All experiments were run on 8 H100 GPUs.

## C TRACE-Edit Construction Details

This section provides the concrete construction details omitted from the main text. The implementation uses structured atomic scene specifications, generates one video per atomic specification, verifies the generated video with a VLM, and then composes only verified atomic videos into relation scenes.

**Object and attribute-value pools.** The current benchmark instantiation samples objects and attribute values from the fixed pools in Tab. 7. Each atomic video contains one object, one attribute type, and one value from the corresponding pool.

| Pool            | Values                                                                                                                                                                                                                                                                         |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objects         | cup, vase, chair, table lamp, backpack, box, bottle, shoe, watch, teapot, sculpture, toy car, pen holder, picture frame, headphones                                                                                                                                            |
| Color values    | deep red, blue, dark green, yellow, warm orange, white, black, silver gray, teal                                                                                                                                                                                               |
| Material values | brushed metal, natural wood, transparent glass, leather, ceramic, canvas, rubber, stone, woven bamboo, wool                                                                                                                                                                    |
| Action values   | static, moving left to right, moving right to left, falling from top to bottom, floating upward from bottom to top, moving from upper-left to lower-right, moving from lower-right to upper-left, moving from lower-left to upper-right, moving from upper-right to lower-left |

Table 7: Object and attribute-value pools used by TRACE-Edit. The target-value probe is evaluated inside each attribute family because the color, material, and action value spaces are disjoint.

**Atomic prompt construction and video generation.** For an atomic specification, the object description is built compositionally. Color and material values are rendered as appearance modifiers, while action values are rendered as motion phrases. For example, the structured tuple ( $o$  = vase,  $a$  = material,  $v$  = transparent glass) yields an object description such as “a transparent-glass vase,”

and ( $o$  = toy car,  $a$  = action,  $v$  = moving left to right) yields “a toy car moving left to right.” The description is then wrapped by one of the following templates:

The center of the frame contains only one {object description}; the subject is clear, the background is simple, and the camera remains stable.  
 One {object description} is located at the center of the frame; the background is clean and the overall motion is natural and stable.  
 The video contains only one {object description}; the subject is complete and clear, and the scene is simple and stable.

Each accepted atomic description is generated as an 81-frame clip at  $480 \times 832$  resolution with the text-to-video generator described in the main paper.

**VLM filtering prompt.** Before composition, each atomic video is verified by a VLM using a structured checklist. The verifier receives the sampled video frames and a prompt of the following form:

Please carefully watch this video and answer each question below. For each item, answer “yes” or “no” and briefly explain the reason. Finally, provide an overall judgment.

1. Does the video contain a unique central subject {object}?
2. Is the {attribute label} of {object} equal to {value}?

Please strictly output JSON only: {"checks": [{"id": 1, "question": "...", "answer": "yes/no", "reason": "..."}, ...], "all\_pass": true/false}.

Only atomic videos for which the parsed field `all_pass` is true are admitted to the verified pool; failed, missing, or unparseable verifier outputs are excluded. The verifier script can also be applied to composed grids by expanding the same checklist over the ordered slots: for each slot  $s \in \{\text{top-left, top-right, bottom-left, bottom-right}\}$ , it asks whether the expected object exists in that slot and whether its attribute value matches the scene specification. In our implementation the verifier uses deterministic decoding, samples 9 frames from each video, and parses the returned JSON decision.

**Grid composition.** Verified atomic videos are grouped by attribute type and value. For each grid scene, we sample one attribute type, select four distinct values from that type, choose one verified atomic video for each value, and randomly assign the four videos to the ordered slots

$$S = (\text{top-left, top-right, bottom-left, bottom-right}).$$

The four clips are synchronized to their common usable length, resized into equal tiles, and pasted frame-by-frame into a  $2 \times 2$  canvas. The grid record stores the ordered slots, the atomic video identifier and path for each cell, each cell’s object metadata, the selected edited slot  $s_e$ , the selected reference slot  $s_r$ , and a `split_group_id` derived from the unordered set of four atomic video identifiers. This group id is used for composition-level train/validation/test splits, preventing the same four-cell visual scene from appearing in multiple partitions.

**Relation-instruction templates and labels.** Each composed scene contributes two directed relation examples: the sampled direction ( $s_e \rightarrow s_r$ ) and the inverted direction ( $s_r \rightarrow s_e$ ). The two examples share the same video but have different edited slots, reference slots, target values, and object-role labels. The instruction templates are:

**Color/material:** Change the color/material of the object at {edited slot} to match the object at {reference slot}. After editing, what is the color/material of the object at {edited slot}?  
**Action:** Make the motion trajectory of the object at {edited slot} consistent with the object at {reference slot}. After editing, what is the motion trajectory of the object at {edited slot}?

For each relation row, the stored labels include `attribute_type`, `source_value`, `target_value`, `edited_slot`, `reference_slot`, `edited_object_name`, `reference_object_name`, `direction_tag`, and the full `scene_spec`. This metadata is what enables the connector-side probes for slot, target value, and object-role variables.



**Dataset scale after construction.** The final dataset contains 5,524 composite videos and 11,048 directed relation examples. Since every grid contributes one forward and one inverted instruction, the relation-direction split is exactly 5,524/5,524. The attribute distribution is 6,490 color, 3,096 material, and 1,462 action examples.

## D VLM Judge Prompt for Output-Level Failure Analysis

This section gives the VLM-as-judge template used for the output-level structural failure analysis in Sec. 5.5. The evaluator receives two videos in a fixed order: the original video before editing and the generated video after editing. We use a two-stage prompt to avoid conflating “no visible edit” with wrong-slot or wrong-binding edits. The implementation uses the Qwen/Qwen3-VL-235B-A22B-Thinking-FP8 judge by default, deterministic decoding with temperature 0, a 4096-token generation limit, samples 9 frames per video, and requests JSON-only outputs. Template variables such as {instruction}, {edited\_side}, and {target\_value} are filled from the relation metadata.

### D.1 Stage 1: edit-activation prompt

The first prompt only asks whether the generated video contains a sufficiently visible edit relative to the input. It intentionally does not judge whether the edit is in the correct slot or has the correct target value.

```
You will see two videos in the following fixed order:
- Video 1: the original video before editing.
- Video 2: the edited/generated result.

Compare the two videos and, using the provided original-scene information and target edit request, judge
only whether a sufficiently visible edit has occurred.

Task background:
- This is a video-editing result verification task.
- In this stage, judge only whether Video 2 contains an obvious, readable, and edit-related change
  compared with Video 1.
- Do not require the visible change to be in the correct slot, on the correct object, or with the correct
  target value.
- If a clear change occurs, even if the slot, object, binding, or target value is wrong, output
  edit_activation_sufficient = true.
- Output edit_activation_sufficient = false only when the overall change is still insufficient. In that
  case, choose an activation_failure_type.

Original scene information:
{scene_text}

Target edit information:
- instruction: {instruction}
- layout_type: {layout}
- attribute_type: {attribute_type}
- edited_side: {edited_side} ({edited_position})
- reference_side: {reference_side} ({reference_position})
- edited_object_name: {edited_object_name}
- reference_object_name: {reference_object_name}
- source_value: {source_value}
- target_value: {target_value}

If edit_activation_sufficient = false, activation_failure_type must be one of:
1. no_visible_change: Video 2 is almost unchanged from Video 1; the target object or target region barely
   changes.
2. partial_or_non_target_change: some change is visible, but it is mainly weak, local, on a non-target
   attribute, on a non-target object, or not sufficient to count as real edit activation.
3. object_missing_or_unreadable: after editing, the target region/object disappears, becomes severely
   blurred, is covered by the background, or cannot be read.

Output rules:
- If edit_activation_sufficient = true, activation_failure_type must be null.
- If edit_activation_sufficient = false, activation_failure_type must be one of the three categories above.
- If the case is genuinely impossible to judge, output null for both fields.

Strictly output JSON only:
{
  "edit_activation_sufficient": true/false/null,
  "activation_failure_type": "no_visible_change/partial_or_non_target_change/object_missing_or_unreadable/
    null",
  "confidence": "high/medium/low",
```

```

"reason_brief": "one sentence explaining whether Video 2 shows a visible edit relative to Video 1 and,
if insufficient, which activation-failure type applies"
}

```

## D.2 Stage 2: structural failure-detail prompt

The second prompt is applied to outputs that are not rejected by the activation stage. It asks whether the visible edit is assigned to the correct slot, object, reference binding, and target value.

```

You will see two videos in the following fixed order:
- Video 1: the original video before editing.
- Video 2: the edited/generated result.

Compare the two videos and, using the provided original-scene information and target edit request, judge
whether the main edit location is correct and whether the target edit is sufficiently clear to
determine the final attribute value.

Task background:
- The previous stage only checked whether Video 2 likely contains a visible edit relative to Video 1.
- This stage checks whether the visible change mainly occurs in the correct slot, on the correct edited
  object, and under the correct reference binding.
- A video may contain a clear change while still editing the wrong slot or object, using the wrong
  reference, or changing the target too weakly to read the final attribute value.
- Therefore, also judge targeted_edit_sufficient. Only when the edit on the correct target object is clear,
  sufficient, and readable should target_correct be judged.
- If targeted_edit_sufficient = false, target_correct must be null.
- If the main problem is slot_correct = false, edited_object_correct = false, or reference_binding_correct
  = false, then targeted_edit_sufficient and target_correct should preferably be null instead of
  forced.
- If the evidence is insufficient for a reliable decision, output null.

Original scene information:
{scene_text}

Target edit information:
- instruction: {instruction}
- layout_type: {layout_type}
- attribute_type: {attribute_type}
- edited_side: {edited_side} ({edited_position})
- reference_side: {reference_side} ({reference_position})
- edited_object_name: {edited_object_name}
- reference_object_name: {reference_object_name}
- source_value: {source_value}
- target_value: {target_value}

Answer the following five structured questions:
1. slot_correct: relative to Video 1, does the main edit change in Video 2 occur at the correct
   edited_side?
2. edited_object_correct: is the object that mainly changed the expected edited_object_name?
3. reference_binding_correct: is the reference object and reference relation understood correctly, without
   confusing the reference object or binding?
4. targeted_edit_sufficient: if the change is in the correct slot and on the target object, is the edit
   clear, sufficient, and stable enough to reliably judge the final attribute value? If the change is
   too weak, too local, occluded, or unreadable, output false. If the previous fields already indicate a
   wrong slot, object, or binding, output null.
5. target_correct: only judge this when targeted_edit_sufficient = true. In that case, has the target
   object in the correct slot changed to target_value for the requested attribute_type? If
   targeted_edit_sufficient is not true, output null.

Strictly output JSON only:
{
  "slot_correct": true/false/null,
  "edited_object_correct": true/false/null,
  "reference_binding_correct": true/false/null,
  "targeted_edit_sufficient": true/false/null,
  "target_correct": true/false/null,
  "confidence": "high/medium/low",
  "reason_brief": "one sentence explaining whether the edit is correct or whether it fails on slot/object/
    binding/target sufficiency/target value"
}

```

## D.3 Failure-label derivation

The structured fields above are normalized before computing the primary failure label. Let `activation_failure_type` be the Stage-1 subtype. The priority order is:

```

if edit_activation_sufficient is false:
    primary_failure = activation_failure_type if available else "under_edit"
elif slot_correct is false:
    primary_failure = "wrong_slot"
elif edited_object_correct is false or reference_binding_correct is false:
    primary_failure = "wrong_object_or_binding"
elif targeted_edit_sufficient is false or under_edit is true:
    primary_failure = "under_edit"
elif target_correct is false:
    primary_failure = "wrong_target_value"
elif all required fields are valid and indicate success:
    primary_failure = "pass"
else:
    primary_failure = "undetermined"

```

This two-stage design is why Sec. 5.5 separates conservative under-editing from structural misassignment: visible but misrouted edits are not counted as under-editing merely because they fail the final task.

## E Nonlinear Probing Experiment

This appendix tests whether the semantic degradation observed in the linear-probe analysis of Sec. 5.2 is merely an artifact of linear separability. We repeat the same probing protocol with a higher-capacity, two-layer MLP probe. The goal is not to replace the linear probe, but to provide a capacity-control diagnostic.

**Experimental setup and metrics.** All feature extraction, task labels, composition-level train/validation/test splits, and train-only PCA preprocessing are kept identical to the linear-probe analysis protocol in Sec. 5.2 and Appendix B. In particular, we use PCA dimension 128, group splits with ratio 0.7/0.15/0.15, and split seeds  $\{0, 1, 42\}$ . The only difference is replacing the linear classifier with a two-layer MLP:

$$f_y^r(h) = \text{softmax}(W_2 \text{Dropout}_{0.2}(\text{GELU}(W_1 \phi_r(h) + b_1)) + b_2), \quad (8)$$

where  $\phi_r(\cdot)$  denotes the same standardized PCA feature used by the linear probe, the hidden width is 256, and the output dimension is the number of classes for label family  $y$ . The MLP is trained with AdamW, learning rate  $10^{-3}$ , weight decay  $10^{-4}$ , batch size 512, maximum 300 epochs, and early stopping patience 30. Accuracy is reported as mean  $\pm$  standard deviation over the three split seeds. As in the main paper, the target-value average is the unweighted mean over action, material, and color target probes.

| Scope                    | Linear pre | Linear post | Linear gap | MLP pre | MLP post | MLP gap |
|--------------------------|------------|-------------|------------|---------|----------|---------|
| All subtasks             | .541       | .523        | -.018      | .559    | .552     | -.007   |
| Excluding attribute type | .465       | .445        | -.021      | .485    | .477     | -.008   |

Table 8: Global comparison between linear and two-layer MLP probes. Values are mean accuracies over the four model cases, seven subtasks, and three split seeds. The gap is post-connector minus pre-connector accuracy.

| Variable          | Chance         | U-Hidden                                      | U-Query                                         | Kiwi-Edit                                       | Wan-Query                                      |
|-------------------|----------------|-----------------------------------------------|-------------------------------------------------|-------------------------------------------------|------------------------------------------------|
| Attribute type    | 1/3            | .999 $\pm$ .001 $\rightarrow$ .996 $\pm$ .001 | 1.000 $\pm$ .001 $\rightarrow$ 1.000 $\pm$ .001 | 1.000 $\pm$ .000 $\rightarrow$ 1.000 $\pm$ .000 | .999 $\pm$ .000 $\rightarrow$ 1.000 $\pm$ .000 |
| Edited slot       | 1/4            | .248 $\pm$ .011 $\rightarrow$ .252 $\pm$ .003 | .952 $\pm$ .002 $\rightarrow$ .812 $\pm$ .013   | .998 $\pm$ .002 $\rightarrow$ .994 $\pm$ .003   | .938 $\pm$ .008 $\rightarrow$ .921 $\pm$ .002  |
| Target value avg. | $\approx$ .107 | .217 $\pm$ .013 $\rightarrow$ .236 $\pm$ .013 | .534 $\pm$ .009 $\rightarrow$ .517 $\pm$ .003   | .527 $\pm$ .010 $\rightarrow$ .497 $\pm$ .005   | .360 $\pm$ .005 $\rightarrow$ .353 $\pm$ .011  |
| Edited object     | 1/15           | .337 $\pm$ .013 $\rightarrow$ .335 $\pm$ .017 | .465 $\pm$ .001 $\rightarrow$ .488 $\pm$ .015   | .568 $\pm$ .006 $\rightarrow$ .545 $\pm$ .002   | .481 $\pm$ .015 $\rightarrow$ .499 $\pm$ .007  |
| Reference object  | 1/15           | .341 $\pm$ .001 $\rightarrow$ .331 $\pm$ .011 | .414 $\pm$ .015 $\rightarrow$ .489 $\pm$ .015   | .625 $\pm$ .010 $\rightarrow$ .612 $\pm$ .004   | .362 $\pm$ .012 $\rightarrow$ .371 $\pm$ .008  |

Table 9: Pre-connector  $\rightarrow$  post-connector two-layer MLP probe accuracy on TRACE-EDIT. The MLP improves absolute accuracy compared with the linear probe, but the main post-connector degradation patterns remain visible.

**Results and analysis.** Tab. 8 shows that the two-layer MLP increases absolute accuracy for both pre- and post-connector representations. The post-connector side benefits slightly more, so the

| Target-value probe | U-Hidden              | U-Query               | Kiwi-Edit             | Wan-Query             |
|--------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| Action target      | .181±.014 → .222±.030 | .279±.013 → .292±.022 | .319±.018 → .274±.011 | .198±.031 → .222±.030 |
| Material target    | .242±.013 → .257±.006 | .634±.024 → .620±.021 | .672±.021 → .649±.023 | .404±.009 → .402±.005 |
| Color target       | .229±.014 → .228±.009 | .687±.012 → .640±.007 | .590±.010 → .567±.017 | .479±.027 → .434±.010 |

Table 10: Full target-value MLP probe accuracy, pre-connector → post-connector. Chance levels are 1/9 for action, 1/10 for material, and 1/9 for color.

average connector gap shrinks from  $-.018$  to  $-.007$  over all subtasks, and from  $-.021$  to  $-.008$  after excluding the nearly saturated attribute-type task. This indicates that some residual information is nonlinearly accessible after connector alignment. However, the average gap does not become a stable positive effect, so the main linear-probe conclusions are not explained away by limited probing model capacity.

The variable-level results in Tab. 9 further support this interpretation. Attribute type remains saturated for every model case, confirming that both linear and nonlinear probes easily recover the coarse edit category. Results on edited-slot task still shows the same structural pattern as the linear probe: U-Query retains a large post-connector drop ( $.952 \rightarrow .812$ ), Wan-Query shows a smaller but consistent drop ( $.938 \rightarrow .921$ ), Kiwi-Edit remains near saturation with only a small gap, and U-Hidden stays near the chance level. Thus, **the key slot-degradation finding is not an artifact of using a linear classifier**.

Target-value results remain more fragile than attribute-type or slot tasks. As shown in Tab. 10, color targets still degrade after connector alignment for U-Query, Kiwi-Edit, and Wan-Query; material targets also remain lower post-connector for U-Query and Kiwi-Edit, and nearly unchanged for Wan-Query. Action targets are weaker and more capacity-sensitive, with small positive gaps for U-Hidden, U-Query, and Wan-Query under the MLP, so we do not treat their sign as a strong conclusion. Rather than that, nonlinear-probe analysis recovers part of the absolute target-value signal, but does not remove the post-connector weakness for the higher-accuracy color and material target probes.

Object-role variables also remain consistent with the linear-probe analysis. U-Query again shows higher post-connector accessibility for edited and reference object names, suggesting that connector alignment can redistribute object information rather than simply erase it. In contrast, Kiwi-Edit still decreases on both object-role probes despite the MLP’s higher absolute accuracy. These patterns reinforce the main paper’s claim that connector alignment selectively reshapes the relational edit program: it preserves coarse intent, leaves some nonlinear residual signal, but still weakens or redistributes the slot–value–role information required for correct editing.

**Takeaway.** The MLP probe should be interpreted as a capacity-control and residual-information diagnostic. It shows that post-connector representations contain additional nonlinearly decodable information, but it does not reverse the main conclusions from the linear probe. Fine-grained structural variables, especially edited slot and target value, remain less reliably accessible after connector alignment than the coarse attribute type.

## F Full Semantic Decodability Results

Tab. 11 expands the target-value column in Tab. 4. Target values are evaluated separately inside each attribute type because action, material, and color have different label spaces.

| Interface | Action target             | Material target           | Color target              |
|-----------|---------------------------|---------------------------|---------------------------|
| U-Hidden  | 0.197±0.012 → 0.196±0.031 | 0.242±0.014 → 0.239±0.016 | 0.221±0.016 → 0.208±0.011 |
| U-Query   | 0.258±0.027 → 0.248±0.007 | 0.619±0.014 → 0.593±0.009 | 0.675±0.013 → 0.633±0.012 |
| Kiwi-Edit | 0.300±0.024 → 0.294±0.018 | 0.623±0.016 → 0.595±0.032 | 0.583±0.009 → 0.566±0.021 |
| Wan-Query | 0.202±0.014 → 0.174±0.014 | 0.390±0.019 → 0.350±0.009 | 0.444±0.008 → 0.394±0.028 |

Table 11: Target-value probe accuracy, pre-connector → post-connector.

## G Feature-View Ablation for U-Query

Tab. 12 shows that text tokens retain strong instruction semantics, while query-only features are weaker and more vulnerable to connector-side slot loss. These are feature-view ablations on the same full extracted features, not separate input-mode runs.

| View       | Attr. type                                    | Edited slot                                   | Material target                               | Color target                                  | Reference object                              |
|------------|-----------------------------------------------|-----------------------------------------------|-----------------------------------------------|-----------------------------------------------|-----------------------------------------------|
| Mixed      | $0.998 \pm 0.002 \rightarrow 0.998 \pm 0.003$ | $0.923 \pm 0.007 \rightarrow 0.739 \pm 0.031$ | $0.619 \pm 0.014 \rightarrow 0.593 \pm 0.009$ | $0.675 \pm 0.013 \rightarrow 0.633 \pm 0.012$ | $0.414 \pm 0.001 \rightarrow 0.444 \pm 0.004$ |
| Text only  | $1.000 \pm 0.000 \rightarrow 0.999 \pm 0.000$ | $0.999 \pm 0.001 \rightarrow 0.988 \pm 0.002$ | $0.897 \pm 0.013 \rightarrow 0.829 \pm 0.011$ | $0.924 \pm 0.005 \rightarrow 0.868 \pm 0.014$ | $0.813 \pm 0.012 \rightarrow 0.595 \pm 0.056$ |
| Query only | $0.999 \pm 0.001 \rightarrow 0.994 \pm 0.003$ | $0.781 \pm 0.016 \rightarrow 0.556 \pm 0.031$ | $0.496 \pm 0.017 \rightarrow 0.528 \pm 0.016$ | $0.551 \pm 0.009 \rightarrow 0.512 \pm 0.001$ | $0.385 \pm 0.014 \rightarrow 0.389 \pm 0.002$ |

Table 12: U-Query feature-view ablation.

## H Token-Level Margin Contribution

Token contribution analysis is run on the edited-slot diagnostic with PCA dimension 256. The table reports mean decision margin, normalized contribution entropy, top-1 positive contribution mass, and the fraction of positive contribution mass assigned to query tokens when query tokens exist.

| Interface | Margin                    | Norm. entropy             | Top-1 mass                | Query positive mass       |
|-----------|---------------------------|---------------------------|---------------------------|---------------------------|
| U-Hidden  | $-1.14 \rightarrow -2.09$ | $0.957 \rightarrow 0.917$ | $0.004 \rightarrow 0.068$ | –                         |
| U-Query   | $18.08 \rightarrow 11.13$ | $0.954 \rightarrow 0.730$ | $0.016 \rightarrow 0.291$ | $0.874 \rightarrow 0.623$ |
| Kiwi-Edit | $24.82 \rightarrow 27.70$ | $0.888 \rightarrow 0.844$ | $0.041 \rightarrow 0.052$ | $1.000 \rightarrow 1.000$ |
| Wan-Query | $16.57 \rightarrow 15.99$ | $0.924 \rightarrow 0.900$ | $0.035 \rightarrow 0.046$ | $0.711 \rightarrow 0.647$ |

Table 13: Token-level contribution summary for edited-slot classification.

## I Detailed Analysis of Attention Mechanisms

This appendix provides a comprehensive quantitative analysis of the internal cross-attention behaviors across three distinct architectural backends (Hidden, Query, and Kiwi). Our probing experiments systematically investigate two primary dimensions: **Token-level Concentration** (the sparsity versus uniformity of attention distributions) and **Group-level Attention Share** (the dynamic routing of attention among different condition modalities).

### I.1 Overall Attention Sparsity and Routing Mechanisms

To evaluate whether the model concentrates its attention on a sparse subset of key condition tokens or distributes it broadly, we compute the Entropy, Gini coefficient, Top- $K$  Mass (the proportion of total attention weight captured by the top  $K$  tokens), and Head Jaccard@16 (the overlap of top-16 tokens across different attention heads). The results are summarized in Tab. 14.

Table 14: Overall Attention Concentration and Head Agreement Across Architectural Backends.

| Backend       | Condition Modality | Entropy ↓ | Gini ↑ | Top-16 Mass ↑ | Top-32 Mass ↑ | Head Jaccard@16 ↑ |
|---------------|--------------------|-----------|--------|---------------|---------------|-------------------|
| <b>Hidden</b> | Vision + Text      | 3.6658    | 0.9254 | 70.58%        | 84.83%        | 0.3989            |
| <b>Query</b>  | Text + Query       | 3.7143    | 0.9286 | 68.04%        | 83.75%        | 0.3958            |
| <b>Kiwi</b>   | Learnable Queries  | 5.6351    | 0.5786 | 13.43%        | 22.88%        | 0.1163            |

The empirical observations reveal a stark contrast in how different architectures route information. The **Kiwi** backend, which relies on a single group of learnable queries, operates akin to a *distributed memory bank*: it exhibits high entropy, low Top- $K$  mass, and minimal overlap between attention heads (Jaccard@16 = 0.1163). This indicates that different attention heads explore distinct, non-overlapping subspaces of the queries. Conversely, the multi-group backends (**Hidden** and **Query**) exhibit highly sparse distributions. In these architectures, the Top-32 tokens cover over 80% of the attention mass, and heads show much higher consensus (Jaccard@16  $\approx$  0.40). This suggests that combining text with visual or query tokens encourages the network to dynamically route information through a sparse bottleneck of critical condition tokens.

## I.2 Spatial-Temporal Handoff in Multi-Group Architectures

For architectures containing multiple groups of condition tokens (Hidden and Query), we further analyze the dynamic shift of attention across different denoising steps (temporal) and network layers (spatial).

Table 15: Dynamic Shift of Group Attention Share in Hidden and Query Backends.

| Backend                   | Dimension                               | Group 1 Share     | Group 2 Share     |
|---------------------------|-----------------------------------------|-------------------|-------------------|
| Hidden<br>(Vision + Text) | <b>Overall Average</b>                  | Vision: 41.35%    | Text: 58.65%      |
|                           | <b>Temporal</b> (Early → Late Steps)    | 39.21% → 43.62%   | 60.79% → 56.38%   |
|                           | <b>Spatial</b> (Dual vs. Single Layers) | 36.45% vs. 46.25% | 63.55% vs. 53.75% |
| Query<br>(Text + Query)   | <b>Overall Average</b>                  | Text: 57.27%      | Query: 42.74%     |
|                           | <b>Temporal</b> (Early → Late Steps)    | 61.57% → 53.09%   | 38.43% → 46.91%   |
|                           | <b>Spatial</b> (Dual vs. Single Layers) | 62.19% vs. 52.34% | 37.81% vs. 47.66% |

As shown in Tab. 15, we observe a clear “handoff” mechanism. Temporally, text tokens predominantly serve as semantic anchors during the early denoising steps, while visual or query tokens incrementally take over the attention share in the later stages. Spatially, the Dual layers heavily favor text semantic alignment, whereas the Single layers shift focus toward visual/query execution. This spatial-temporal handoff implies that the generation process inherently follows a coarse-to-fine paradigm: text establishes the global semantic layout early on and in shallower layers, while visual/query tokens execute fine-grained structural and material editing later in the process.

## I.3 Architectural Determinism Across Editing Attributes

Finally, we verify the robustness of these attention patterns by evaluating across different editing attributes (Action, Material, Color).

Table 16: Core Attention Metrics Across Different Editing Attributes.

| Backend | Attribute Type | Core Group Share      | Entropy | Top-16 Mass |
|---------|----------------|-----------------------|---------|-------------|
| Hidden  | Action         | Vision: 41.10%        | 3.7068  | 69.66%      |
|         | Material       | Vision: 40.13%        | 3.6309  | 71.41%      |
|         | Color          | Vision: <b>42.89%</b> | 3.6462  | 70.98%      |
| Query   | Action         | Query: 42.21%         | 3.7390  | 67.23%      |
|         | Material       | Query: 42.66%         | 3.6862  | 68.92%      |
|         | Color          | Query: <b>43.51%</b>  | 3.7095  | 68.26%      |

Interestingly, as shown in Tab. 16, the attention concentration and group share metrics remain remarkably stable regardless of the specific editing instruction. Although color editing relies slightly more on the Vision/Query groups, the overall variance is marginal. This confirms a principle of *architectural determinism*: the observed routing bottlenecks and attention sparsity are inherent properties of the model’s structural design and alignment strategy, rather than artifacts of specific prompt semantics. Consequently, resolving multimodal alignment bottlenecks requires fundamental architectural interventions rather than merely augmenting training data with specific attribute types.

## J Output Failure Counts

Table 17 details the absolute counts of various failure modes observed during our evaluation. Note that all percentage rates reported in the main paper are calculated exclusively based on these successfully evaluated outputs.

| Interface | Pass | Under | Wrong slot | Wrong binding | Wrong target | Undet. |
|-----------|------|-------|------------|---------------|--------------|--------|
| U-Hidden  | 30   | 15    | 38         | 31            | 3            | 2      |
| U-Query   | 31   | 17    | 46         | 40            | 4            | 1      |
| Kiwi-Edit | 52   | 20    | 35         | 21            | 6            | 3      |
| Wan-Query | 34   | 15    | 31         | 46            | 8            | 1      |

Table 17: Absolute counts of evaluation outcomes. Each interface was tested on 144 generated examples. Outputs that were unavailable or rejected by the evaluator are excluded from the rate computations in the main text.

### J.1 Additional Qualitative Results

In this section, we present further qualitative comparisons on complex multi-region video editing tasks, utilizing a  $2 \times 2$  grid layout. These highly challenging scenarios evaluate the models’ capacity to execute localized edits while preventing “concept bleeding” into non-target regions.

As illustrated in Fig. 4, given the instruction “*Change the color of the picture frame in the bottom-right of the video to match that of the backpack in the top-left,*” the models are required to modify the bottom-right region. However, **U-Query** and **Wan-Query** incorrectly target the wrong region, changing the color of the backpack to yellow while leaving the picture frame unchanged. **Kiwi-Edit** correctly targets the bottom-right frame but fails to match the silver color, altering it to a wooden texture instead. Meanwhile, **U-Hidden** struggles to apply any noticeable edit (under-editing).

Fig. 5 highlights common failure modes under the instruction “*Change the material of the toy car in the top-left of the video to match that of the sculpture in the bottom-left.*” **U-Query** suffers from severe object disappearance, as the target toy car completely vanishes from the scene. Conversely, **Wan-Query** exhibits extreme over-editing and concept bleeding, drastically altering the materials and colors of all non-target objects (e.g., the sculpture, boot, and bottle) to a uniform ceramic-like texture. Furthermore, **U-Hidden** incorrectly alters the non-target sculpture instead of the car. These findings underscore the ongoing challenge of balancing precise instruction adherence with the preservation of non-target identities in complex compositional scenes.



Instruction: "Change the color of the picture frame in the bottom-right of the video to match that of the backpack in the top-left."



Figure 4: Qualitative comparison of localized editing. The instruction targets the bottom-right picture frame. Both *U-Query* and *Wan-Query* incorrectly edit the top-left backpack instead, while *Kiwi-Edit* targets the correct region but fails to achieve the desired silver color.

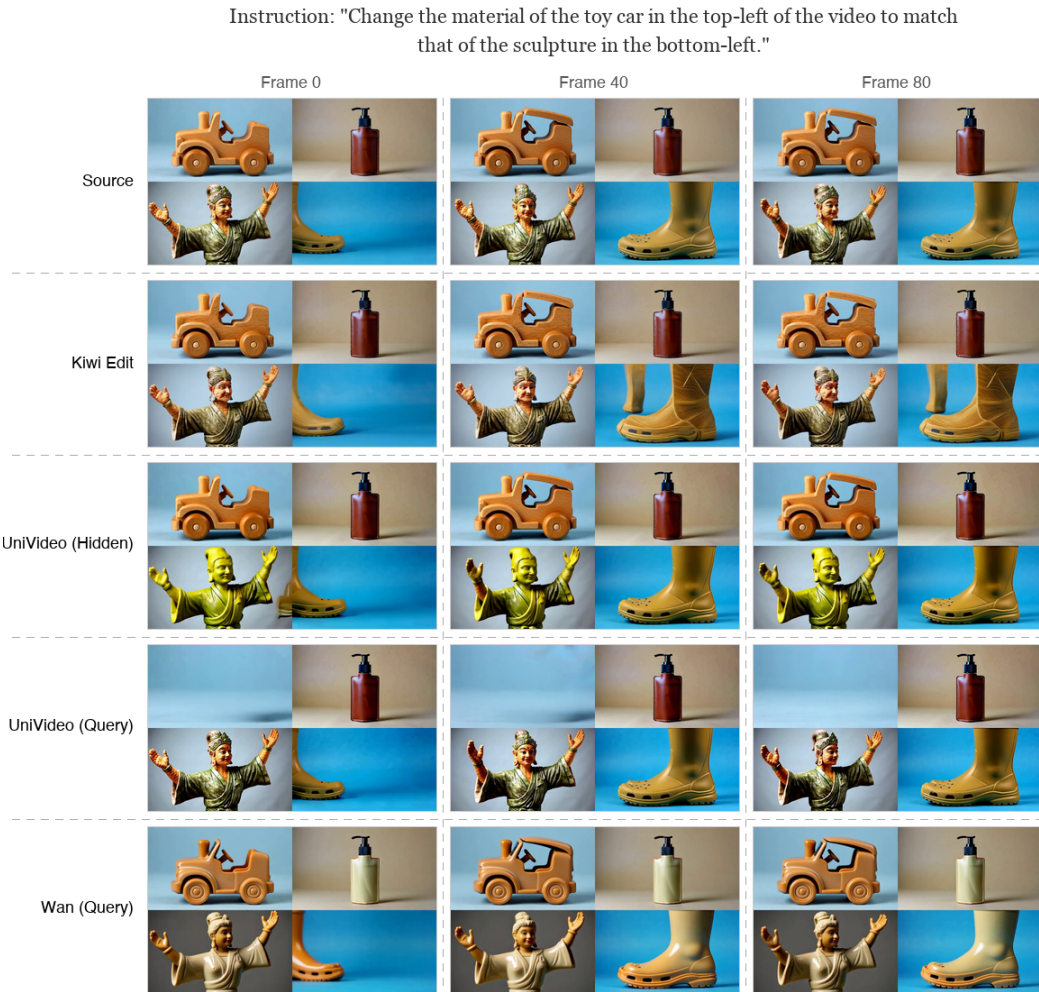


Figure 5: Failure cases in localized material editing. *U-Query* suffers from object disappearance (the top-left car vanishes), and *Wan-Query* exhibits severe over-editing, altering the materials of non-target objects across all quadrants.